Frontend Advanced Practice - Complete Task List

📦 PART A: React + TypeScript Challenges (25 Tasks)

Section 1: Advanced Data Handling (10 Tasks)

Task 1: Nested Folder Tree View

Goal: Display a folder/file structure like VS Code explorer

- Create a recursive `TreeNode` component
- Show folders with expand/collapse icons (▶/▼)
- Display files with different icons based on type
- Add depth-based indentation (padding-left)
- Handle click to select/highlight a node
- **TypeScript:** Define `TreeItem` type with `children?: TreeItem[]`

Task 2: Paginated Product Table with Filters

Goal: Fetch products with pagination, sorting, and filters combined

- Fetch from DummyJSON `/products` API
- Add filters: category dropdown, price range slider, search by name
- Add sorting: price (asc/desc), rating
- Show "Showing 1-20 of 100" pagination info
- Update URL params on filter/sort change
- **TypeScript:** Define `ProductFilter` and `SortOption` types

Task 3: Async Search Dropdown

Goal: Search GitHub users with debounced API calls

- Use GitHub API `/search/users?q={query}`
- Show dropdown results as you type (min 2 chars)
- Debounce API calls (300ms delay)
- Show loading spinner inside dropdown
- Handle "No results found" state

- Click to select and close dropdown

Task 4: Infinite Scroll with Deduplication

Goal: Load posts infinitely without duplicate items

- Fetch from JSONPlaceholder `/posts?_page=X&_limit=10`
- Detect scroll bottom using `IntersectionObserver`
- Show loading spinner at bottom while fetching
- Prevent duplicate posts if same page loads twice
- Add "No more posts" message at end
- **TypeScript:** Type safe `useInfiniteScroll` hook

Task 5: Flat Array to Nested Tree Converter

Goal: Transform comment array into nested replies

- Input: `[{id:1, parentId:null}, {id:2, parentId:1}]`
- Output: Nested structure with `children` array
- Display as indented comment thread
- Show reply count for each comment
- **TypeScript:** Generic `arrayToTree<T>` utility function

Task 6: Combined Search + Filters + Tabs

Goal: Reusable component with all filter logic

- Create `<DataView>` component accepting config
- Tabs: "All", "Active", "Archived"
- Search bar with real-time filtering
- Multi-select category filter
- One component controls all state
- **TypeScript:** Use generics for flexible data type

Task 7: Dynamic Reorderable Table

Goal: Table where columns can be hidden/reordered via drag

- Display users in table with: Name, Email, Phone, Company
- Add column visibility toggles (checkboxes)
- Drag column headers to reorder
- Save column order in localStorage
- **TypeScript:** `ColumnConfig<T>` type with accessor keys

Task 8: Search Keyword Highlighter

Goal: Highlight matched text in search results

- Fetch posts and search by title/body
- Wrap matching keywords in `<mark>` tags
- Handle multiple keywords (split by space)
- Case-insensitive matching
- **TypeScript:** `highlightText(text: string, query: string): JSX.Element`

Task 9: Event Timeline

Goal: Display events sorted by date with visual timeline

- Fetch events from API with date field
- Sort by date (newest first)
- Show vertical timeline with dots and lines
- Format dates nicely (e.g., "2 days ago")
- **TypeScript:** `Event` type with `Date` field

Task 10: Virtualized Large List

Goal: Render 10,000 items smoothly using `react-window`

- Use `FixedSizeList` from react-window
- Render only visible items (viewport)
- Add search/filter (re-render filtered list)
- Show total count vs visible count
- **TypeScript:** Type `FixedSizeList` props correctly

Section 2: TypeScript + Utility Types (6 Tasks)

Task 11: Deep Merge Objects

Goal: Create type-safe object merger

```typescript
function deepMerge<T, U>(obj1: T, obj2: U): T & U
```

- Recursively merge nested objects
- Handle arrays (concatenate or replace)
- Write tests for edge cases
- **Challenge:** Handle conflicting types safely

Task 12: Complex Form State Types

Goal: Define reusable form types with validation

```typescript
type FormState<T> = {
  values: T;
  errors: Partial<Record<keyof T, string>>;
  touched: Partial<Record<keyof T, boolean>>;
}
```

- Use `Record`, `Partial`, `Pick`
- Create helper: `getFieldError(field: keyof T)`
- Type-safe form field names

Task 13: useLocalStorage Hook

Goal: Generic localStorage hook with full TS support

```typescript
function useLocalStorage<T>(key: string, initialValue: T): [T, (value: T) => void]
```

- Serialize/deserialize JSON automatically
- Handle parsing errors gracefully

- Sync across tabs using `storage` event
- **TypeScript:** Infer type from initial value

Task 14: Type-Safe Route Config

Goal: Define routes with metadata and params

```typescript
type Route = {
  path: string;
  component: React.ComponentType;
  meta: { title: string; requiresAuth: boolean };
}
```

- Create routes array with autocomplete
- Generate route paths with params
- Type-safe navigation helper

Task 15: Generic Table Component

Goal: Reusable table with dynamic columns

```typescript
type Column<T> = {
  key: keyof T;
  header: string;
  render?: (value: T[keyof T], row: T) => JSX.Element;
}
```

- Accept `data: T[]` and `columns: Column<T>[]`
- Type-safe column accessors
- Optional custom cell renderer

Task 16: Discriminated Union Notifications

Goal: Type-safe notification system

```typescript
```

```
type Notification =
  | { type: 'success'; message: string }
  | { type: 'error'; message: string; retry: () => void }
  | { type: 'info'; message: string; link?: string }
```

- TypeScript enforces correct props per type
- Render different UI based on type
- Toast component with union type

Section 3: Form Power Tasks (6 Tasks)

Task 17: Multi-Step Form Wizard

Goal: Form split into 3 steps with progress save

- Steps: Personal Info → Address → Review
- Save each step to localStorage
- Show progress bar (1/3, 2/3, 3/3)
- Validate before allowing next step
- Load saved progress on refresh

Task 18: Conditional Form Fields

Goal: Show/hide fields based on other field values

- Country dropdown (if "Bangladesh" → show divisions)
- Payment method radio (if "Card" → show card fields)
- Dynamic validation rules per condition
- Reset conditional fields when parent changes

Task 19: Backend Validation Errors

Goal: Display inline errors from API response

- Submit form to mock API
- API returns `{ errors: { email: "Already exists" } }`
- Show error under specific field (not generic message)

- Clear error when user edits field
- Distinguish client vs server errors

Task 20: File Upload with Progress

Goal: Drag-drop file upload with progress bar

- Accept image files (jpg, png, max 5MB)
- Show preview thumbnail
- Display upload progress (0-100%)
- Use `XMLHttpRequest` or `fetch` with progress events
- Cancel upload option

Task 21: Pre-fill and Edit Form

Goal: Load user data, edit, and submit updates

- Fetch user from API on mount
- Pre-fill all form fields
- Track which fields changed (dirty state)
- PATCH only changed fields on submit
- Show "Saved successfully" message

Task 22: Keyboard Navigation

Goal: Navigate form using Tab and Enter

- Auto-focus first field on mount
- Tab moves to next field (custom order)
- Enter submits form (if not textarea)
- Shift+Tab goes backwards
- Skip disabled fields

Section 4: Component Design Challenges (3 Tasks)

Task 23: Responsive Navbar

Goal: Desktop dropdown menu, mobile hamburger

- Desktop: Hover dropdowns for nested menus
- Mobile: Hamburger icon → slide-in menu
- Close on outside click or route change
- Active link highlighting
- Smooth transitions

Task 24: Weekend-Disabled Date Picker

Goal: Custom date picker blocking weekends

- Render calendar grid (7 cols × 5 rows)
- Disable Saturday and Sunday
- Highlight today and selected date
- Navigate between months
- Return selected date to parent

Task 25: Theme Switcher

Goal: Light/Dark/Custom themes with persistence

- Context provider for theme state
- Toggle button with icon (sun/moon)
- Save preference to localStorage
- Apply theme via CSS variables or Tailwind
- Custom theme: user picks accent color

🧩 PART B: Third-Party Library Challenges (25 Tasks)

Section 5: Form Libraries (5 Tasks)

Task 26: Event Registration with React Hook Form

Goal: Multi-field form with validation

- **Libraries:** `react-hook-form`, `zod`, `react-datepicker`, `react-toastify`
- Fields: Name, Email, Phone, Date of Birth, Event Date

- Validation: Email format, age 18+, future event date
- Show error messages inline
- Success toast on submit

Task 27: Dynamic Product Form with Formik

Goal: Add/remove product variants dynamically

- **Libraries:** formik, yup, shadcn/ui
- Array of variants (size, price, stock)
- Add/remove variant rows
- Validate each variant
- Submit as nested object

Task 28: File Upload to Cloudinary

Goal: Image upload with preview and cloud storage

- **Libraries:** react-dropzone, axios, cloudinary
- Drag-drop or click to upload
- Show image preview before upload
- Upload to Cloudinary API
- Store returned URL in form
- Handle upload errors (size, format)

Task 29: Multi-Step Checkout Form

Goal: Wizard form for checkout flow

- **Libraries:** react-hook-form, zod, framer-motion
- Steps: Cart → Shipping → Payment
- Animate step transitions
- Validate before proceeding
- Submit all data at end

Task 30: Form with Real-Time Validation

Goal: Show validation as user types

- **Libraries:** react-hook-form, zod
- Username: Check availability via API
- Password: Show strength meter
- Confirm password: Real-time match check
- Debounce async validations

Section 6: Data Fetching & State (5 Tasks)

Task 31: Product Dashboard with TanStack Query

Goal: Fetch, filter, update products

- **Libraries:** @tanstack/react-query, zustand, shadcn/ui
- Fetch from DummyJSON /products
- Add loading, error, empty states
- Optimistic update on quantity change
- Persist filters in Zustand

Task 32: Real-Time Search with React Query

Goal: Search movies with caching

- **Libraries:** @tanstack/react-query, TMDB API
- Search as you type (debounced)
- Cache results (don't refetch same query)
- Show loading skeleton
- Navigate to movie detail page

Task 33: Infinite Scroll with TanStack Query

Goal: Load more items infinitely

- **Libraries:** @tanstack/react-query, react-intersection-observer
- Use useInfiniteQuery
- Detect bottom with IntersectionObserver

- Flatten pages for display
- "Load More" button fallback

Task 34: Optimistic UI Updates

Goal: Instant feedback before API confirms

- **Libraries:** `@tanstack/react-query`
- Like/unlike post instantly
- Rollback if API fails
- Show error toast on failure
- Use `useMutation` with `onMutate`

Task 35: Global State with Zustand

Goal: Manage cart state across pages

- **Libraries:** `zustand`, `zustand/middleware/persist`
- Add/remove items from cart
- Update quantity
- Calculate total price
- Persist cart in localStorage

Section 7: UI Component Libraries (5 Tasks)

Task 36: Authentication Flow with Clerk

Goal: Sign up, sign in, protected routes

- **Libraries:** `@clerk/nextjs`, `shadcn/ui`
- Sign in/sign up pages
- Protect dashboard routes
- Show user profile info
- Sign out functionality

Task 37: Modal Dialog with Radix UI

Goal: Accessible modal with focus trap

- **Libraries:** `@radix-ui/react-dialog`, `shadcn/ui`
- Open/close modal
- Focus trap inside modal
- Close on Escape or backdrop click
- Animate with Framer Motion

Task 38: Dropdown Menu with Radix UI

Goal: Accessible dropdown with keyboard nav

- **Libraries:** `@radix-ui/react-dropdown-menu`
- Trigger button opens menu
- Arrow key navigation
- Close on click outside
- Nested submenus

Task 39: Toast Notifications

Goal: Show success/error/info toasts

- **Libraries:** `react-toastify` or `sonner`
- Trigger from anywhere in app
- Auto-dismiss after 3s
- Different styles per type
- Stack multiple toasts

Task 40: Tabs Component

Goal: Tabbed interface with routing

- **Libraries:** `@radix-ui/react-tabs`, Next.js router
- Switch between Profile, Settings, Billing tabs
- Sync with URL query params
- Animate tab content transitions

- Accessible with keyboard

Section 8: Advanced Integrations (5 Tasks)

Task 41: Weather Dashboard

Goal: Fetch and display weather by city

- **Libraries:** `@tanstack/react-query`, OpenWeather API, `lucide-react`
- Search city by name
- Show current weather + 5-day forecast
- Display weather icons
- Toggle °C / °F units
- Save recent searches (Zustand)

Task 42: GitHub Profile Viewer

Goal: Display user info and repositories

- **Libraries:** `@tanstack/react-query`, GitHub API
- Search user by username
- Show profile: avatar, bio, followers, repos count
- List repositories with stars/forks
- Navigate to repo detail page

Task 43: Currency Converter

Goal: Convert between currencies with live rates

- **Libraries:** `@tanstack/react-query`, Exchange Rate API
- Two dropdowns: From/To currency
- Input amount, show converted value
- Update on currency or amount change
- Cache rates (valid for 1 hour)

Task 44: Google Maps Location Picker

Goal: Display map with user location

- **Libraries:** `@react-google-maps/api`
- Show map centered on user's location
- Drop a marker on click
- Reverse geocode (lat/lng → address)
- Search places autocomplete

Task 45: Movie Database with TMDB

Goal: Browse movies with genre filter

- **Libraries:** `@tanstack/react-query`, TMDB API, `framer-motion`
- Fetch trending movies
- Genre filter dropdown
- Click movie → detail page
- Show cast, rating, trailer link
- Animate page transitions

Section 9: Animation & UX (3 Tasks)

Task 46: Page Transitions with Framer Motion

Goal: Smooth animations between pages

- **Libraries:** `framer-motion`, Next.js
- Fade in/out on route change
- Slide animations for mobile menu
- Stagger list items on load
- Exit animations before unmount

Task 47: Skeleton Loaders

Goal: Show placeholder UI while loading

- **Libraries:** `shadcn/ui` Skeleton
- Show skeleton for cards, table, profile

- Match actual content shape
- Fade in real content when loaded
- Pulse animation for skeleton

Task 48: Drag and Drop List

Goal: Reorder items via drag-drop

- **Libraries:** `dnd-kit`
- Drag to reorder todo items
- Visual feedback while dragging
- Save new order to state
- Persist order to API/localStorage

Section 10: Accessibility & Performance (2 Tasks)

Task 49: Focus Management & ARIA

Goal: Make custom components accessible

- **Libraries:** `@radix-ui` primitives
- Modal: Focus trap, close on Escape
- Dropdown: Arrow key navigation
- Add `aria-label`, `aria-expanded`
- Keyboard-only navigation test

Task 50: Code Splitting & Lazy Loading

Goal: Optimize bundle size

- **Libraries:** React.lazy, Vite/Next.js
- Lazy load heavy components (Chart, Map)
- Route-based code splitting
- Show loading fallback
- Measure bundle size reduction

🎯 How to Practice

1. **Pick 2-3 tasks per week**
2. **Read docs first** (don't copy-paste blindly)
3. **Build in CodeSandbox or local**
4. **Compare with others' solutions**
5. **Deploy to Vercel/Netlify**

✅ Success Checklist Per Task

- [ ] Component works as expected
- [ ] TypeScript has no errors
- [ ] Responsive design (mobile + desktop)
- [ ] Loading/error states handled
- [ ] Accessible (keyboard navigation)
- [ ] Code is clean and commented
- [ ] Git commit with clear message

You got this! 🔥 One task at a time, you'll master everything.